

ITCS386 Project Assignment 1

Unit Test for Open-Source Software Projects

Score: 15% out of 100% of the total course score

Item	GitHub	In-Class
Format	Create a folder for your project called <your_team_name>_project1	
Submission Date	23 September 2026 (before 23:55) The code must be pushed to GitHub (use the commit time)	
Q&A		24 September 2026

Instructions

In this project assignment, you will use the knowledge of software testing and quality assurance, specifically test case design, input space partitioning, unit testing, and automated testing, to improve an existing open-source software project.

Please follow the steps below.

1. Each team picks one of the available Java open-source projects on GitHub (pick the one that has only a few tests so that you can improve the software).

Examples:

- 1) <https://github.com/jfree/jfreechart>
- 2) <https://github.com/apache/tomcat>

The chosen project must:

- 1) Be an actual software project, **not a tutorial nor a training document**.
 - 2) Have at least 20 stars.
 - 3) It should be written in Java.
 - 4) Must not be the same as other groups.
2. Study the existing testing framework within the project (e.g., Ant, Maven, Gradle). You will need to create an automated test using the existing automated testing framework available in the project.
3. Develop 10 additional unit test suites for the project. **The test suites must not duplicate the existing ones.** One unit test case represents a testing method in your test suite that is mapped to a specific tested method in the project. You can have many methods containing one assert statement in each method or one method with several assert statements.

For example, the code below is a test suite with 3 test cases.

```
@Test
public void testAdd1() {
```

```
assertEquals(2, add(1, 1));
assertEquals(0, add(-2, 2));
assertEquals(-2, add(-5, 3));
}
```

Or, you can separate each test case into a method as shown below.

```
@Test
public void testAdd1() {
    assertEquals(2, add(1, 1));
}

@Test
public void testAdd2() {
    assertEquals(0, add(-2, 2));
}

@Test
public void testAdd3() {
    assertEquals(-2, add(-5, 3));
}
```

4. Make sure your unit tests can be executed automatically using the project's testing framework, such as running the test suite via Ant, Maven, or Gradle. You must show this execution using the project's testing framework during the project presentation.
5. Also, add prefix values or postfix values if needed (using `@Before` and `@After` in JUnit).
6. Document the unit tests that you have created in README.md. For each test case, define the following:
 - a. Name of the test case, e.g., `testNumZeroEmptyArray`
 - b. The goal of the test case is to verify that the created array contains no elements.
 - c. The characteristics for this test case were developed using techniques from Input Space Partitioning. You must create ****at least**** one interface-based characteristic and ****at least**** one functionality-based characteristic for each test suite.
7. In each test case, document the steps for input domain modelling in the README.md file, following the guidelines outlined in the Week 3 and Week 4 slides.
 - a. Identify testable functions.
 - b. Identify parameters, return types, return values, and exceptional behaviour.
 - c. Model the input domain.
 - d. Combine partitions to define test requirements - use the 5 approaches to find combinations of values (ACoC, ECC, PWC, BCC, MBCC). You must apply the 5 approaches to your 10-unit test cases (thus, each approach will be used in 2-unit test cases).
 - e. Derive test values and expected values. **These are the values that you must use when you implement test cases in JUnit.**

- f. Please ensure you use the test values designed in steps 6 and 7 when writing unit test cases in Java.
 - g. You can combine interface-based and functionality-based characteristics' blocks when creating test case combinations.
8. Create your unit test cases in a separate class file, distinct from the existing test classes. Name it appropriately to fit within the project's context. However, at the top of the file, put the following comment.

```
/* Copyright (C) <Year> <Full Name (can be multiple persons) - All
Rights Reserved
 * You may use, distribute and modify this code under the terms of
the XYZ license.
 */
```

For example, the licensing statement using the MIT license can be written as follows.

```
/* Copyright (C) 1883 Thomas Edison - All Rights Reserved
 * You may use, distribute and modify this code under the terms of
the MIT license.
 */
```

9. Make sure that you add sufficient explanation on how each test case is derived. Your work will be graded based on the content in the README.md file. It is better to be verbose than terse!
10. Also, make sure you put everything in the GitHub repo. The instructor will review the created test cases prior to the Q&A session.

Project Presentation & QA

The project presentation and Q&A are for you to present your work and for the instructor to ask questions about it. Each group will have about 20 minutes for the presentation and 10 minutes for Q&A. Every team member must be present for the project presentation and Q&A, and each member must give a presentation. Any absence will result in a 30% deduction of your project score (individually).

1. The presentation should include an explanation of the selected project for testing, along with details of each of your 10 test suites.
2. Show the execution of the complete test suite using the automated testing framework. Your test cases must be included in the test suites.
3. Answer any questions the instructor may have.

Grading Criteria

	Scoring Criteria		
Item	Low	Medium	High
Project selection (in terms of its suitability and difficulty)	0.5 (Very small and simple project, only)	1 (Large project with a number of actual users)	2 (Widely used open-source projects)

	Scoring Criteria		
	a few users)		
Quality of the created test cases (1 point for each test case)	0-5 (Some of the test cases are missing OR The 10 test cases have been created; however, they do not adhere to the ISP concepts.	6-9 (The 10 test cases are created with ISP, but no sufficient explanation).	10 (The 10 test cases are carefully created by following the ISP concepts with a full detailed explanation)
Automated test case execution	0 (Cannot execute automated test cases during the demo, OR run the automated test cases without using the project's existing build system)		1 (Successfully execute automated test cases by using the project's existing build system)
Quality of the writing (README.md)	0 (No content in the README.md)	0.5 (Content exists in the README.md, but not very well-written)	1 (Content in the README.md is well-written, complete, and easy to understand)
Q&A	0 (Cannot answer any questions)	0.5 (Can answer some of the questions correctly)	1 (Can answer all the questions successfully)

Change Log

Item	Date/Time
Initial version	6 August 2026 5:33